

Classification binaire de nodules pulmonaires sur radiographies thoraciques

Architecture EfficientNetV2–MEAM avec prétraitement SEPE

Clément B.

2026

Table des matières

1	Introduction	3
2	Construction de l’entrée : représentation à trois canaux	3
3	Prétraitement SEPE	4
3.1	Rehaussement des hautes fréquences	4
3.2	Masque de poids adaptatif	4
3.3	Régularisation laplacienne	4
3.4	Implémentation GPU (version batch)	4
4	Architecture du modèle	5
4.1	Vue d’ensemble	5
4.2	Backbone : EfficientNetV2-S	6
4.3	Module MEAM : attention multi-échelle	6
4.3.1	Alignement spatial	6
4.3.2	Projections Q, K, V	6
4.3.3	Attention croisée inter-échelles	6
4.3.4	Fusion finale	7
4.4	Tête de classification	7
5	Stratégie d’entraînement	7
5.1	Séparation des données	7
5.2	Gestion du déséquilibre de classes	7
5.3	Augmentation de données	8
5.4	Focal Loss	8

5.5	Optimisation	8
5.6	Critère de sauvegarde	8
6	Évaluation	9
7	Résultat	9
8	Localisation du nodule	9
8.1	Tête de régression (NoduleLocator)	9
8.2	Données de localisation	10
8.3	Entraînement en deux phases	10
9	Résumé de l'architecture	10

1 Introduction

Ce document décrit l’architecture et le fonctionnement du pipeline de classification binaire de nodules pulmonaires à partir de radiographies thoraciques. Le système repose sur trois composantes principales :

1. un prétraitement par rehaussement adaptatif des contours (**SEPE** — *Sparse Edge-Preserving Enhancement*),
2. un mécanisme d’attention multi-échelle (**MEAM** — *Multi-Scale Embedding Attention Mechanism*),
3. un backbone **EfficientNetV2-S** pré-entraîné sur ImageNet, couplé à une tête de classification linéaire.

Le jeu de données provient du *NIH Chest X-ray Dataset*, filtré pour ne conserver que deux classes : **Nodule** et **No Finding**.

2 Construction de l’entrée : représentation à trois canaux

Chaque radiographie thoracique en niveaux de gris est transformée en un tenseur à **trois canaux** complémentaires, conçus pour fournir au réseau des informations de nature différente :

Canal	Nom	Description
c_1	Densité brute	Image originale normalisée par Min-Max dans $[0, 1]$. Préserve l’information de densité radiologique.
c_2	SEPE	Rehaussement adaptatif des contours (section 3). Calculé sur GPU lors de l’entraînement via <code>_build_3ch_batch</code> .
c_3	CLAHE	<i>Contrast Limited Adaptive Histogram Equalization</i> (<code>clipLimit = 2.0</code> , grille 8×8). Améliore le contraste local dans les régions à faible dynamique.

TABLE 1 – Canaux d’entrée du modèle ($512 \times 512 \times 3$).

Le **Dataset** ne charge que deux canaux sur CPU (c_1 et c_3) afin d’éviter un goulot d’étranglement. Le canal SEPE (c_2) est calculé sur GPU juste avant l’inférence grâce à la version batch du préprocesseur.

3 Prétraitement SEPE

Le *Sparse Edge-Preserving Enhancement* rehausse les structures fines (contours de nodules, bronches) tout en limitant l'amplification du bruit. Il se décompose en trois étapes.

3.1 Rehaussement des hautes fréquences

Soit $I(x, y)$ l'image d'entrée et $G_\sigma = G_\sigma * I$ sa version lissée par un noyau gaussien d'écart-type σ . Le détail haute fréquence $I - G_\sigma$ est pondéré par un masque adaptatif W :

$$I_{\text{enh}}(x, y) = I(x, y) + \alpha W(x, y) (I(x, y) - G_\sigma(x, y)) \quad (1)$$

où $\alpha = 1,5$ contrôle l'intensité du rehaussement.

3.2 Masque de poids adaptatif

Le masque W est construit à partir de la magnitude du gradient, de façon à *préserver* les bords existants (gradient fort $\Rightarrow W \rightarrow 0$) et *rehausser* les zones lisses (gradient faible $\Rightarrow W \rightarrow 1$) :

$$W(x, y) = \exp\left(-\frac{\|\nabla I(x, y)\|^2}{\sigma_e^2}\right) \quad (2)$$

avec $\sigma_e = 0,1$ qui règle la sensibilité aux gradients. Le gradient est estimé par les filtres de Sobel 3×3 .

3.3 Régularisation laplacienne

Pour empêcher l'amplification du bruit haute fréquence, un terme de régularisation basé sur le laplacien discret est soustrait :

$$I_{\text{SEPE}}(x, y) = I_{\text{enh}}(x, y) - \lambda |\nabla^2 I(x, y)| \quad (3)$$

avec $\lambda = 0,1$. Le résultat est ensuite ramené dans $[\min(I), \max(I)]$ par clipping.

3.4 Implémentation GPU (version batch)

Lors de l'entraînement, le SEPE est calculé sur GPU par convolutions PyTorch (`F.conv2d`) pour traiter un batch complet en parallèle. Les opérations reproduisent exactement les équations (1)–(3) avec :

- un noyau gaussien 2D séparable de taille $\lceil 6\sigma \rceil + 1$,
- les filtres de Sobel 3×3 pour le gradient,

— le noyau laplacien $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$.

Le canal SEPE est ensuite normalisé par division par le maximum du batch afin de le ramener dans $[0, 1]$.

Paramètre	Valeur	Rôle
σ	1,0	Écart-type du lissage gaussien
α	1,5	Facteur d'amplification des détails
σ_e	0,1	Sensibilité du masque de poids
λ	0,1	Coefficient de régularisation laplacienne

TABLE 2 – Hyperparamètres du préprocesseur SEPE.

4 Architecture du modèle

4.1 Vue d'ensemble

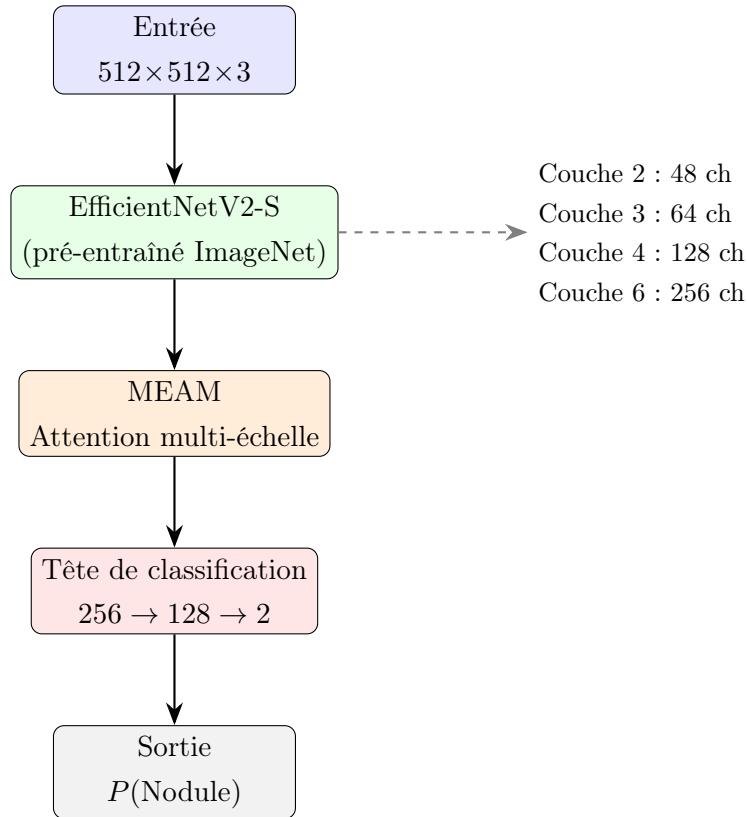


FIGURE 1 – Architecture globale du classificateur NoduleClassifier.

4.2 Backbone : EfficientNetV2-S

Le backbone est un **EfficientNetV2-S** pré-entraîné sur ImageNet-1K. Seule la partie *features* (sans la tête de classification d’origine) est conservée. Les cartes de caractéristiques sont extraites à quatre niveaux intermédiaires :

Indice de bloc	Canaux	Résolution relative
2	48	$\times \frac{1}{4}$
3	64	$\times \frac{1}{8}$
4	128	$\times \frac{1}{16}$
6	256	$\times \frac{1}{32}$

TABLE 3 – Points d’extraction multi-échelle pour le variant **s**.

Le variant **s** (Small) est utilisé pour des contraintes de VRAM.

4.3 Module MEAM : attention multi-échelle

Le *Multi-Scale Embedding Attention Mechanism* fusionne les quatre cartes de caractéristiques extraites du backbone via un mécanisme d’attention croisée inter-échelles.

4.3.1 Alignement spatial

Toutes les cartes sont d’abord redimensionnées par interpolation bilinéaire à la résolution de la plus petite ($H_{\min} \times W_{\min}$, celle du bloc 6).

4.3.2 Projections Q, K, V

Pour chaque échelle $s \in \{1, \dots, S\}$ (avec $S = 4$), trois convolutions 1×1 projettent la carte $\mathbf{F}_s$ vers les espaces query, key et value :

$$\mathbf{Q}_s = W_s^Q \mathbf{F}_s \in \mathbb{R}^{B \times d_k \times HW} \quad (4)$$

$$\mathbf{K}_s = W_s^K \mathbf{F}_s \in \mathbb{R}^{B \times d_k \times HW} \quad (5)$$

$$\mathbf{V}_s = W_s^V \mathbf{F}_s \in \mathbb{R}^{B \times d_v \times HW} \quad (6)$$

avec $d_k = d_v = 64$ par défaut.

4.3.3 Attention croisée inter-échelles

Pour chaque paire d’échelles (i, j) , un score d’attention est calculé :

$$A_{ij} = \text{softmax}\left(\frac{\mathbf{Q}_i^\top \mathbf{K}_j}{\sqrt{d_k}}\right) \quad (7)$$

La sortie de l'échelle i est obtenue en sommant les contributions de toutes les échelles :

$$\hat{\mathbf{F}}_i = \sum_{j=1}^S \mathbf{V}_j A_{ij}^\top \quad (8)$$

4.3.4 Fusion finale

Les S sorties sont concaténées le long de la dimension des canaux, puis projetées par une convolution 1×1 suivie d'une *Batch Normalization* et d'une activation ReLU :

$$\mathbf{F}_{\text{fused}} = \text{ReLU}(\text{BN}(W^{\text{out}} [\hat{\mathbf{F}}_1 \parallel \dots \parallel \hat{\mathbf{F}}_S])) \in \mathbb{R}^{B \times 256 \times H_{\min} \times W_{\min}} \quad (9)$$

4.4 Tête de classification

La carte de caractéristiques fusionnée passe par :

1. **Adaptive Average Pooling** $\rightarrow$ vecteur de dimension 256,
2. couche linéaire $256 \rightarrow 128 + \text{ReLU}$,
3. **Dropout** ($p = 0,5$) pour la régularisation,
4. couche linéaire $128 \rightarrow 2$ (logits des deux classes).

La probabilité de la classe **Nodule** est obtenue par application de la fonction Softmax sur les logits : $P(\text{Nodule}) = \text{softmax}(\mathbf{z})_1$.

5 Stratégie d'entraînement

5.1 Séparation des données

Le jeu de données est divisé en deux sous-ensembles par un split **stratifié 80/20** (`train_test_split` de scikit-learn, `random_state=42`), garantissant que la proportion de nodules est identique dans les deux ensembles.

5.2 Gestion du déséquilibre de classes

Le jeu de données NIH présente un fort déséquilibre entre les classes **Nodule** (minoritaire) et **No Finding** (majoritaire). Deux mécanismes sont combinés :

- **WeightedRandomSampler** : sur-échantillonne la classe minoritaire. Le nombre d'échantillons par époque est fixé à $2 \times n_{\text{nodules}}$ pour équilibrer les classes.
- **Focal Loss** (section 5.4) : réduit la contribution des exemples faciles au gradient.

5.3 Augmentation de données

Appliquée uniquement à l'ensemble d'entraînement via la bibliothèque `Albumentations` :

Transformation	Probabilité	Paramètres
Retournement horizontal	0,50	—
Shift / Scale / Rotation	0,55	$\pm 5\%$ décal., $\pm 5\%$ échelle, ± 10
Luminosité / Contraste	0,45	$\pm 10\%$
Déformation élastique	0,40	$\alpha = 120$, $\sigma = 6$, affine = 3,6

TABLE 4 – Pipeline d'augmentation de données.

5.4 Focal Loss

La *Focal Loss* [?] modifie la cross-entropie standard en la pondérant par un facteur $(1 - p_t)^\gamma$ qui réduit la perte pour les prédictions confiantes :

$$\mathcal{L}_{\text{FL}}(p_t) = -(1 - p_t)^\gamma \log(p_t) \quad (10)$$

où p_t est la probabilité prédite pour la classe correcte et $\gamma = 2,0$.

5.5 Optimisation

Composant	Configuration
Optimiseur	AdamW ($\text{lr} = 10^{-4}$, $\text{weight_decay} = 10^{-3}$)
Scheduler	Cosine Annealing ($T_{\text{max}} = \text{nombre d'époques}$)
Gradient clipping	Norme max = 1,0
Taille de batch	12
Résolution	512×512
Époques	50
Seed	42

TABLE 5 – Hyperparamètres d'entraînement.

5.6 Critère de sauvegarde

Le modèle est sauvegardé lorsque l'**Average Precision** (AP) sur l'ensemble de validation atteint un nouveau maximum. Deux fichiers sont produits :

- `dernier_train.pth` — poids du backbone (`model.backbone.state_dict()`), réutilisable pour une tâche de localisation,
- `best_dernier_train.pth` — modèle complet (`model.state_dict()`).


```

--- Seuil optimal calculé (0.566) ---
Vrais Négatifs (TN): 11299 | Faux Positifs (FP): 779
Faux Négatifs (FN): 61    | Vrais Positifs (TP): 475
Sensibilité : 0.8862 | Spécificité : 0.9355

```

FIGURE 2 – Résultat du modèle

6 Évaluation

L'évaluation du modèle s'effectue via :

- la **courbe ROC** et l'**AUC**,
- le **seuil optimal** par l'indice de Youden ($\arg \max(\text{TPR} - \text{FPR})$),
- la **matrice de confusion** à différents seuils de décision (0,5 ; seuil optimal),
- la **sensibilité** $= \frac{TP}{TP+FN}$ et la **spécificité** $= \frac{TN}{TN+FP}$.

7 Résultat

Le modèle obtient 88,62 % de recall sur la classe nodule, cela peut être amélioré à 93,6 % en baissant le seuil et augmentant légèrement le nombre de faux positif (le faisant passé à 1068). Il reste tout de même à noter que le modèle a une précision sur la classe nodule relativement faible. Cela pourrait être compenser par un déséquilibre de classe moins important et plus d'image de nodule afin de généralisé.

8 Localisation du nodule

Une première approche d'interprétabilité par **Grad-CAM** (*Gradient-weighted Class Activation Mapping*) a été explorée afin de visualiser les zones exploitées par le réseau. Les résultats se sont toutefois avérés **peu concluants** : les activations étaient diffuses sur l'ensemble de la radiographie plutôt que concentrées autour du nodule, en raison de la résolution spatiale très réduite de la dernière couche (16×16) et de la petite taille des nodules relativement à l'image. Ce constat a motivé le développement d'une approche supervisée dédiée à la localisation.

8.1 Tête de régression (NoduleLocator)

Le module NoduleLocator réutilise le **backbone** et le **MEAM** pré-entraînés pour la classification, sur lesquels il greffe une **tête de régression** qui prédit les coordonnées normalisées $(x, y) \in [0, 1]^2$ du nodule dans l'image :

$$\hat{\mathbf{p}} = \sigma(\text{FC}_{128 \rightarrow 2}(\text{Dropout}_{0,4}(\text{ReLU}(\text{FC}_{256 \rightarrow 128}(\text{GAP}(\mathbf{F}_{\text{MEAM}})))))) \quad (11)$$

où GAP désigne le *Global Average Pooling* et σ la fonction sigmoïde qui contraint la sortie dans $[0, 1]^2$.

8.2 Données de localisation

Un fichier CSV fournit pour chaque image les coordonnées (x, y) en pixels du centre du nodule. Elles sont normalisées dans $[0, 1]$ après redimensionnement de l'image à 512×512 .

8.3 Entraînement en deux phases

L'entraînement adopte un schéma en **deux phases** afin de préserver les représentations du classificateur :

1. **Phase 1 — Warm-up** (~ 5 époques) : backbone et MEAM *gelés*, seule la tête de régression s'entraîne ($\text{lr} = 10^{-3}$, AdamW).
2. **Phase 2 — Fine-tuning spatial** (~ 10 époques) : le MEAM est *dégelé* avec un taux différencié (MEAM : 10^{-5} ; tête : 10^{-4}). Le backbone reste gelé.

La fonction de perte est la **Smooth L1 Loss** et l'erreur de localisation est mesurée en pixels : $e_{\text{px}} = \|\hat{\mathbf{p}} - \mathbf{p}\|_2 \times 512$.

9 Résumé de l'architecture

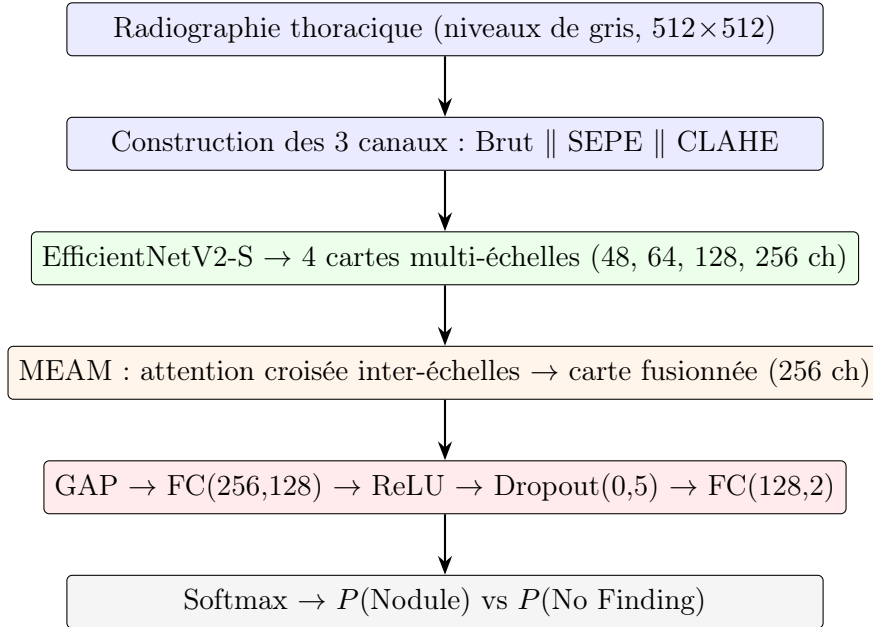


FIGURE 3 – Pipeline complet, de l'image brute à la prédiction.

Références

- [1] M. Dhayalini, B. Revathi alias Ponmozhi, *Multi-phase deep learning framework with Multiscale Adaptive Swin Transformer and embedding attention for precision lung nodule detection and classification*, Scientific Reports, vol. 16, article 1652, 2025. doi:10.1038/s41598-025-31147-2
- [2] S. Raza, R. Zia, I. A. Usmani, N. A. Almujaally, N. Alasbali, M. Hanif, *Trans RCED-UNet3+ : a hybrid CNN-transformer model for precise lung nodule segmentation*, Frontiers in Oncology, vol. 15, article 1654466, 2025. doi:10.3389/fonc.2025.1654466